



TD 03 - Docker

Docker est une plateforme open-source qui permet de créer, déployer et exécuter des applications dans des conteneurs isolés. Ce TD vous apprendra à manipuler les bases de la gestion des conteneurs.

Objectifs

À l'issue de ce TD, vous serez capable de :

1. Installer Docker sur le système d'exploitation de votre machine.
2. Apprendre à utiliser les commandes de gestion des images et des conteneurs Docker.
3. Créer une image Docker à partir d'un fichier Dockerfile.

PRÉ-REQUIS

1. Connaissance de base en Spring Boot et des commandes shell.
2. Un environnement de travail prêt avec Git, Java (JDK 17 minimum) et un IDE (VS Codium).

Installer Docker

Docker est déjà installé sur les PC de l'école. Sur votre machine, installez *Docker Desktop* en suivant ce [lien](#). Une fois installé, n'oubliez pas de vérifier que Docker fonctionne via la commande `docker --version`.

DOCKER ENGINE

C'est le moteur principal de Docker. Il permet de créer, exécuter et gérer des conteneurs. Avec Docker Engine :

- Vous utilisez le daemon Docker (`dockerd`) pour gérer les conteneurs.
- Vous contrôlez Docker via la ligne de commande (**CLI**).

❗ DOCKER DESKTOP

C'est une application complète. Elle inclut Docker Engine, d'autres outils comme Docker Compose ainsi qu'une interface graphique pour gérer vos conteneurs.

Registre de conteneurs - Container registry

Les conteneurs utilisés par Docker peuvent être créés localement ou téléchargés d'une plateforme en ligne appelée **registre de conteneurs**. Le registre utilisé par défaut par Docker pour télécharger (**pull**) ou publier (**push**) ces images est **Docker Hub**. Vous y trouverez des images officielles maintenues par Docker et sa communauté.

i IMAGE DOCKER

Une image Docker est un **modèle** qui contient tout ce dont une application a besoin pour s'exécuter : le code, les bibliothèques, les dépendances, les configurations et le système d'exploitation minimal requis. Elle sert de point de départ pour créer des conteneurs.

Consultez le site de [Docker Hub](#) et cherchez quelles sont les versions de l'image ubuntu disponibles au téléchargement. La version 24.04 est-elle disponible ?

Le début de l'apprentissage de Docker consiste en la maîtrise des commandes de base pour gérer des images. Parcourez ce tutoriel et prenez note des différentes commandes.

⚠ ATTENTION

Sur les PC de l'école, le démon n'est pas lancé par défaut. Pour que les commandes `docker` ci-dessous fonctionnent, il faut d'abord exécuter Docker Desktop qui va se charger de lancer le démon.

Nous vous demandons toutefois d'éviter d'utiliser l'interface graphique. Vous serez plus performant en apprenant les lignes de commandes et ce sera absolument nécessaire pour écrire des scripts d'automatisation des tâches.

⚠ ATTENTION

Les commandes qui vont suivre peuvent télécharger de grosses quantités de données. Attention si vous utilisez une connexion 5G.

❗ TUTORIEL IMAGE DOCKER

1. Vérifiez qu'aucune image n'est présente sur votre machine via la commande `docker image ls`.
2. Téléchargez l'image de la version 24.04 d'ubuntu trouvée sur le registre avec la commande `docker pull ubuntu:24.04`.
3. Vérifiez la présence de l'image sur votre machine avec la commande `docker image ls`.
4. **Notez la taille** de cette image en MB.
5. **Notez l'identifiant** de cette image.
6. Effacez l'image via la commande `docker rmi <identifiant de l'image>`.
7. Listez à nouveau les images pour vérifier le succès de la suppression.
8. Cherchez, téléchargez et **comparez la taille** de l'image de la dernière version de Alpine Linux (`alpine:latest`).

Conteneur Docker - Docker Container

i CONTENEUR DOCKER

Un conteneur est une instance en cours d'exécution d'une image Docker. C'est une unité isolée qui regroupe une application et tout son environnement d'exécution, y compris les dépendances, les bibliothèques et les configurations nécessaires.

Un premier conteneur : Hello World

Comme pour les images, la première étape pour comprendre les conteneurs est d'introduire les commandes associées. Suivez ce tutoriel et prenez notes des différentes commandes.

❗ TUTORIEL CONTENEUR DOCKER

1. Exécutez la commande `docker run --name devops-hello ubuntu:24.04 echo 'Hello World!'`.
2. Vérifiez que la commande a bien affiché `Hello World` dans le terminal.
3. Consultez la liste des conteneurs via `docker ps -a`.
4. Comparez le résultat de la commande précédente avec le résultat de la commande `docker ps`. Que constatez-vous ?

5. Vérifiez dans les logs du conteneur, via la commande `docker logs devops-hello`, que l'instruction a bien été exécutée.
6. Démarrez à nouveau le conteneur via `docker start devops-hello`.
7. Consultez à nouveau les logs et vérifiez la seconde exécution de l'instruction.
8. Consultez le **statut** du conteneur via `docker ps -a`. Le conteneur est-il en cours d'exécution ?

Si vous décomposez la commande qui a permis d'afficher `Hello World`, vous pouvez y lire trois parties :

- exécution par Docker du conteneur de la version 24.04 de ubuntu `docker run ubuntu:24.04`
- exécution **à l'intérieur du conteneur** de la commande `echo 'Hello World!'`
- association du nom `devops-hello` avec le conteneur via `--name devops-hello`

L'exécution de la commande `docker run` précédente affiche dans le terminal le message suivant :

```
Unable to find image 'ubuntu:24.04' locally
24.04: Pulling from library/ubuntu
de44b265507a: Pull complete
Digest:
sha256:80dd3c3b9c6cecb9f1667e9290b3bc61b78c2678c02cbdae5f0fea92cc6734ab
Status: Downloaded newer image for ubuntu:24.04
Hello world!
```

Ce message retrace les étapes réalisées par Docker pour exécuter le conteneur :

1. Docker vérifie si l'image demandée est disponible localement comme si vous exécutiez `docker image ls`.
2. S'il ne la trouve pas, il télécharge l'image depuis Docker Hub.
3. Les couches de l'image sont téléchargées individuellement (un `Pull` par couche) .
4. Une fois l'image prête, Docker exécute la commande `echo 'Hello World!'` dans un conteneur basé sur cette image.
5. Le résultat de la commande est affiché (`Hello World!`) avant que **le conteneur ne se termine**.

Remarquez que l'ordre des paramètres est important. `docker run ubuntu:24.04 echo --name devops-hello 'Hello World!'` conduira à une erreur.

⚠ CONTENEUR ANONYME

Vous pouvez exécuter un conteneur anonyme `docker run ubuntu:24.04 echo 'Hello World!'`. Il faudra simplement récupérer son nom via `docker ps -a`.

A l'intérieur d'un conteneur

Ce premier conteneur est un peu décevant car il s'arrête aussi tôt créé. Suivez le prochain tutoriel qui utilise un conteneur qui reste allumé 30 minutes afin que nous puissions le manipuler un peu plus.

❗ TUTORIEL CONTENEUR DOCKER ACTIF

1. Exécutez la commande `docker run --name devops-sleep ubuntu:24.04 sleep 1800`.
2. **Ouvrez un second terminal** et consultez le statut du conteneur via la commande `docker ps -a`.
3. Entrez **DANS** le conteneur via la commande `docker exec -it devops-sleep bash`.
 - i. Listez les fichiers/dossiers présents dans le conteneur `ls -l`.
 - ii. Déplacez-vous dans le dossier `/home/ubuntu` via la commande `cd`.
 - iii. Essayez de cloner le dépôt `git clone https://git.esi-bru.be/4dop1dr-ressources/demo-no-db.git`.
 - iv. `git` n'est pas installé dans ce conteneur ubuntu, installez le via `apt update` et `apt -y install git`.
 - v. Essayez à nouveau de cloner le dépôt `git clone https://git.esi-bru.be/4dop1dr-ressources/demo-no-db.git`.
 - vi. Sortez du conteneur via `exit`.
4. Arrêtez le conteneur via `docker stop devops-sleep`.
5. Vérifiez le statut du conteneur via `docker ps -a`.

La commande pour "entrer dans le conteneur" `docker exec -it devops-sleep bash` peut se décomposer comme suit :

- `docker exec` est une commande utilisée pour exécuter un processus dans un conteneur en cours d'exécution.
- `-i` (interactive) : Maintient l'entrée standard (stdin) ouverte pour que vous puissiez interagir avec le conteneur.

- `-t` (tty) : Alloue un pseudo-terminal pour permettre une expérience interactive (comme un terminal).
- `devops-sleep` est le nom du conteneur dans lequel la commande sera exécutée.
- `bash` est la commande qui sera exécutée à l'intérieur du conteneur. Il s'agit de l'interpréteur de commandes, pour interagir avec le conteneur via un shell.

Mise à jour d'une image

Le conteneur `devops-sleep` n'est plus le même que suite à sa création basée sur l'image d'origine. Vous pouvez décider d'enregistrer ces changements dans une nouvelle image afin de disposer d'une image ubuntu avec git déjà installé.

Commencez par noter l'identifiant du conteneur que vous avez utilisé. Cette identifiant est affiché via la commande `docker ps -a`. Ensuite exécutez la commande :

```
docker commit -a 'g12345' -m "Installation de git dans l'image"
<identifiant_conteneur> ajout_de_git
```

Consultez la nouvelle image créée via `docker image ls` et prenez attention au **nom** de cette image et comparez la **taille** de cette image avec celle de l'image ubuntu d'origine.

Vous pouvez consultez le détail de cette nouvelle image via la commande `docker history ajout_de_git`.

Créer une image via un Dockerfile

Après avoir exploré `docker commit` pour créer une image à partir d'un conteneur en cours d'exécution, vous allez découvrir une approche plus reproductible, l'utilisation d'un Dockerfile, qui permet d'automatiser la création d'images de manière déclarative. Ce fichier texte contient une série d'instructions permettant de construire une image Docker.

La liste des instructions (directives) est disponible via la documentation mais voici celles que nous allons utiliser

- `FROM` : Spécifie l'image de base.
- `RUN` : Exécute des commandes pendant la création de l'image.
- `COPY` : Ajoute des fichiers locaux dans l'image.
- `CMD` : Définit la commande exécutée lorsque le conteneur démarre.

- **LABEL** : ajoute des métadonnées à l'image.
- **ENV** : définit des variables d'environnement accessibles pendant la construction de l'image et au moment de l'exécution du conteneur.
- **EXPOSE** : indique au Dockerfile quels ports le conteneur utilisera pour communiquer avec l'extérieur.
- **WORKDIR** : définit le répertoire courant dans lequel les instructions suivantes du Dockerfile ou les commandes exécutées dans le conteneur, s'exécuteront. Ce répertoire est créé s'il n'existe pas déjà.

Conteneuriser un fichier jar

Commencez votre apprentissage des Dockerfiles en conteneurisant une application Java sous forme de fichier .jar. L'objectif est de créer une image Docker capable d'exécuter une application de manière portable et isolée.

EXERCICE 1 : CONTENEURISER UN JAR

Modifiez le Dockerfile ci-dessous pour conteneuriser votre application `demo-no-db`. Ce premier Dockerfile doit copier le fichier `demo-1.0.0.jar` dans le conteneur. La commande pour construire une image à partir d'un fichier intitulé *Dockerfile* présent dans le repertoire courant est `docker build -f Dockerfile -t g12345/spring-demo-no-db .` (le `.` fait partie de la commande)

Dockerfile

```
1  # Image de base Ubuntu 24.04
2  FROM ubuntu:24.04
3  # Nom de L'auteur
4  LABEL author="g12345"
5
6  # Mise à jour des paquets et installation de Java 17
7  # Chaque instruction RUN ajoute une couche à L'image Docker.
8  # En combinant les commandes dans une seule instruction RUN,
9  # vous limitez le nombre de couches inutiles.
10 RUN apt-get update && \
11     apt-get install -y openjdk-21-jre && \
12     apt-get clean
13
14 # Définition des variables d'environnements pour Java
15 ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
16 ENV PATH=$JAVA_HOME/bin:$PATH
```

```
17
18 # Création d'un répertoire de travail pour l'application
19 WORKDIR /app
20
21 # Copie du fichier JAR de la machine locale vers le conteneur
22 COPY your-application.jar /app/your-application.jar
23
24 # Expose le port utilisé par l'application
25 EXPOSE 8080
26
27 # Commande pour exécuter l'application
28 CMD ["java", "-jar", "/app/your-application.jar"]
29
```

Vérifiez vos modifications en démarrant un conteneur basé sur l'image `g12345/spring-demo-no-db` et consommez le service grâce à l'url `localhost:8080/config`.

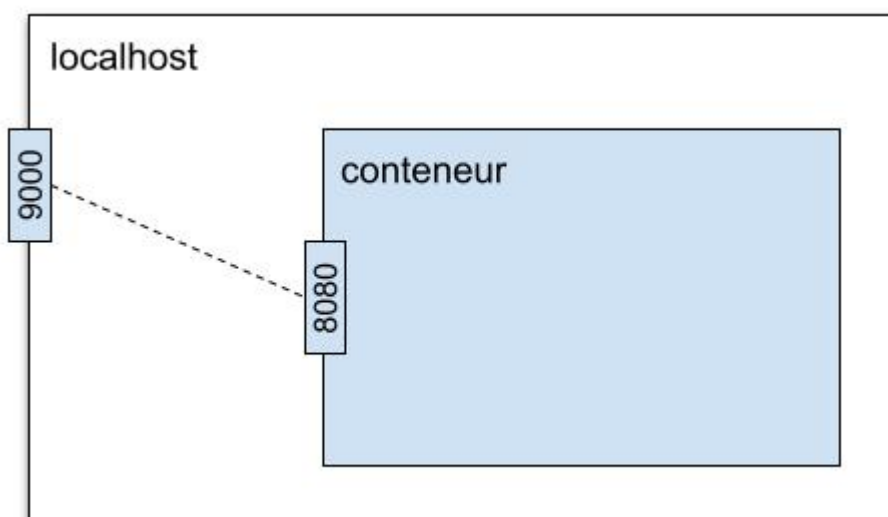
Ca ne fonctionne pas ? C'est normal !

Le serveur web tourne dans un conteneur que vous pouvez voir comme une machine virtuelle. Elle écoute sur le port 8080 du conteneur, pas de votre machine. Pour le moment, rien n'écoute sur le port 8080 du *localhost*. Pour que ça fonctionne, il faut créer un lien entre un port de votre machine et le port du conteneur. On parle de *redirection de port*.

❗ TUTORIEL REDIRECTION DE PORT

Lancez cette fois le conteneur avec une option : `docker run -p 9000:8080 g12345/spring-demo-no-db` et testez la consommation du service.

L'option permet justement de définir la redirection de port.



En pratique, on utilisera probablement aussi 8080 comme port local. Ici, nous avons pris une valeur fort différente pour bien mettre en évidence la différence entre les deux ports.

Essayez maintenant avec une option supplémentaire : `docker run -d -p 9000:8080 g12345/spring-demo-no-db`. Qu'apporte-t-elle ?

i EXERCICE 2 : UTILISER UNE SOURCE IMAGE ADAPTÉE

1. Modifiez le Dockerfile en vous basant sur l'image `eclipse-temurin` (`FROM`).
2. Vérifiez sur Docker Hub si cette image contient une version de Java.
3. Supprimez les instructions devenues inutiles.

Ajouter l'étape d'empaquetage au Dockerfile

Lorsqu'on automatise le déploiement d'une application, on doit souvent effectuer l'empaquetage (la création du jar dans notre cas) au sein du conteneur. Essayez de vous entraîner à cette pratique avec l'exercice suivant.

i EXERCICE 3 : EMPAQUETER DANS LE CONTENEUR

Recherchez sur Docker Hub l'image officielle de maven. Modifiez le Dockerfile précédent pour se baser sur cette image de maven. Ce Dockerfile doit copier les sources et le pom.xml de votre application dans le conteneur et empaqueter l'application au sein du conteneur grâce à maven.

Vérifiez vos modifications en consommant le service fourni par le conteneur construit par cette image.

Optimisation des ressources

Utiliser une image multi-stage, où la compilation est faite dans une première étape et seule l'application compilée est copiée dans l'image finale permet d'optimiser les ressources. Pour réaliser un tel Dockerfile vous devez :

- définir une première image pour la compilation
- créer une seconde image pour l'exécution

Essayez-vous à cette pratique avec l'exercice ci-dessous.

i EXERCICE 4 : OPTIMISER UNE IMAGE AVEC UNE BUILD MULTI-STAGE

Améliorez le Dockerfile précédent pour utiliser une multi stage build.

- La commande `FROM` possède une option `AS nom_image` pour nommer en interne l'image créée.
- Une deuxième commande `FROM` dans le fichier débute une deuxième image.
- Dans la deuxième image la commande `COPY` peut référencer un fichier de la première image grâce au nom qui lui a été donné : `COPY --from=nom_image ...`.

Comparez les tailles des différentes images produites dans ces exercices afin de déterminer la pratique la plus efficace en ressources.

Persister des données avec les volumes

Vous allez manipuler un conteneur MySQL avec un volume Docker pour persister les données même après la suppression du conteneur.

Variables d'environnement

Pour passer des variables d'environnement lors de l'exécution d'un conteneur, utilisez l'option `-e` :

```
docker run -e NOM_VARIABLE=valeur -e AUTRE_VARIABLE=valeur <image>
```

Vous pouvez également placer vos variables d'environnement dans un fichier et les utiliser via :

```
docker run --env-file .env <image>
```

i EXERCICE 5 : UN CONTENEUR MYSQL SANS VOLUME

1. Cherchez sur Docker Hub l'image officielle de MySQL.
2. Notez le nom des variables d'environnement à utiliser pour démarrer un tel conteneur :
 - nom de l'utilisateur non root
 - mot de passe de l'utilisateur non root
 - mot de passe du root
 - nom de la base de données créée automatiquement au démarrage du conteneur

3. Démarrez un conteneur MySQL avec ces différentes variables.

Exécution de requêtes dans un conteneur

Pour créer une table dans une base de données MySQL, l'outil en ligne de commande `mysql` s'utilise comme suit :

```
mysql -u g12345 -psecret -D mydatabase -e "CREATE TABLE person (MATRICULE INT PRIMARY KEY, NAME VARCHAR(100));"
```

- `-u` spécifie le nom d'utilisateur pour se connecter à la base de données.
- `-p` spécifie le mot de passe de l'utilisateur.
- `-D` spécifie la base de données à utiliser.
- `-e` spécifie requête SQL à exécuter.

i EXERCICE 6 : CHARGEMENT DES DONNÉES

Grâce à la commande `docker exec` et à l'outil en ligne de commande `mysql`, créez dans le conteneur MySQL la table `person` et insérez-y deux personnes. Vérifiez le résultat en exécutant la requête `SELECT * FROM person`.

Supprimez ensuite ce conteneur.

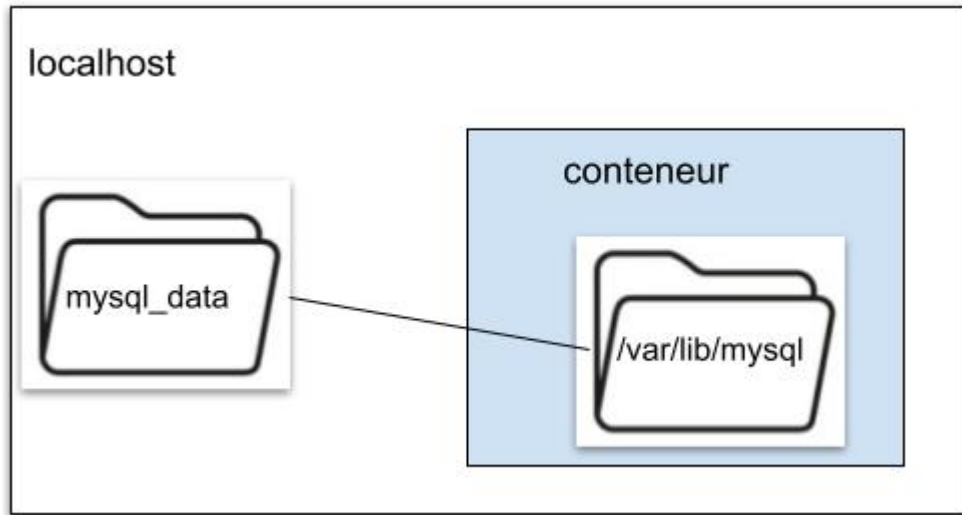
Création du volume

Afin de ne pas perdre les données de la base de données lors de la suppression d'un conteneur, vous allez attacher un volume à ce conteneur via la commande :

```
docker run -v <chemin_hôte>:<chemin_conteneur> <image>
```

! LA NOTION DE VOLUME

Créer un volume c'est créer un lien entre un dossier sur la machine locale et un dossier dans



un conteneur.

Par défaut, *mysql* crée ses bases de données dans le dossier `/var/lib/mysql`. Avec l'option `-v`, on peut indiquer que ce dossier est en fait le dossier `mysql_data` sur le disque dur local. L'information sera ainsi préservée même en cas de destruction du conteneur.

i EXERCICE 7 : CRÉATION D'UN VOLUME DOCKER

1. Créez un dossier intitulé `mysql_data`.
2. Lancez un conteneur MySQL en utilisant comme volume le dossier hôte `mysql_data` et le dossier du conteneur `/var/lib/mysql`.
3. Vérifiez que le dossier `mysql_data` n'est pas vide. Si ce dossier est vide, pensez à utiliser le **chemin absolu** du dossier hôte `mysql_data`.
4. Créez une table `person` dans la base de données.
5. Insérez des données dans cette table.
6. Vérifiez les données insérées en affichant le contenu de la table `person`.
7. Supprimez le conteneur MySQL, puis recréez-en un nouveau en réutilisant le même volume.
8. Vérifiez si les données ont été conservées après la suppression et recréation du conteneur en affichant le contenu de la table `person`.

User non root

Vous constatez que vous n'avez pas les droits en écriture sur le contenu du dossier `mysql_data`. Les fichiers qui ont été créés le sont avec l'utilisateur d'exécution du conteneur, c'est à dire, `root`. Il existe plusieurs solutions pour palier à ce problème. Par exemple vous pouvez demander à Docker d'exécutez la commande `chmod` pour changer les droits du dossier `/var/lib/mysql` :

```
docker exec -ti --user root <conteneur_mysql> chmod -R 777 /var/lib/mysql
```

Pour en savoir plus sur les bonnes pratiques concernant la gestion des utilisateurs dans une image Docker,

[consultez cet article.](#)